

Data Warehousing

Modelo dimensional y ETL

IIC2413 · Semana 15

Agenda

1. ¿Por qué no usamos una BD relacional para análisis?
2. OLTP vs. OLAP
3. El modelo estrella (*star schema*)
4. Del esquema normalizado al modelo dimensional
5. "¿No estamos violando BCNF?" - oops
6. ETL: Extract, Transform, Load
7. El cubo de datos y las operaciones OLAP

Pregunta para empezar

Supongamos que somos la empresa detrás de una plataforma de ventas online. Tenemos este esquema (bien normalizado, BCNF):

```
Clientes(id_cliente, nombre, email, ciudad, id_región)
Regiones(id_región, nombre, país)
Productos(id_producto, nombre, id_categoría, precio_base)
Categorías(id_categoría, nombre, id_familia)
Familias(id_familia, nombre)
Tiendas(id_tienda, nombre, ciudad, id_región)
Ventas(id_venta, id_cliente, id_producto, id_tienda, fecha, cantidad, monto)
```

¿Cuánto vendimos por región y categoría el último trimestre?

El costo de esa consulta

```
SELECT r.nombre AS región, f.nombre AS familia, SUM(v.monto)
FROM Ventas v
JOIN Clientes c    ON v.id_cliente = c.id_cliente
JOIN Regiones r    ON c.id_región  = r.id_región
JOIN Productos p    ON v.id_producto = p.id_producto
JOIN Categorías ca  ON p.id_categoria = ca.id_categoria
JOIN Familias f     ON ca.id_familia = f.id_familia
WHERE v.fecha BETWEEN '2025-10-01' AND '2025-12-31'
GROUP BY r.nombre, f.nombre;
```

- **5 joins** sobre una tabla de hechos con millones de filas
- Se ejecuta varias veces al día para distintos reportes
- La BD tiene que soportar también escrituras de órdenes en tiempo real

Hay tensión entre las necesidades de **escritura transaccional** y **lectura analítica**.

OLTP (Online Transaction Processing) vs. OLAP (Online Analytical Processing)

	OLTP	OLAP
Propósito	Operaciones del negocio	Análisis e inteligencia
Consultas	Muchas, pequeñas, por llave	Pocas, enormes, agregaciones
Escrituras	Frecuentes, críticas	Raras (batch)
Diseño óptimo	Normalizado (BCNF/3NF)	Desnormalizado
Ejemplo	Registrar una venta	¿Qué región vende más en invierno?

La solución: separar los dos sistemas.

Un **Data Warehouse (DW)** es una base de datos orientada a análisis, separada de los sistemas transaccionales, diseñada específicamente para OLAP.

¿Necesito OLTP u OLAP? Una guía práctica

Señal	→ OLTP	→ DW/OLAP
Acceso por llave primaria	✓	—
Agregaciones sobre millones de filas	—	✓
Escrituras en tiempo real (ms)	✓	—
Reportes históricos y dashboards	—	✓
Muchas transacciones concurrentes	✓	—
Análisis de tendencias (meses, años)	—	✓
Schema cambia seguido	✓	—
Consultas ad-hoc sobre todas las filas	—	✓

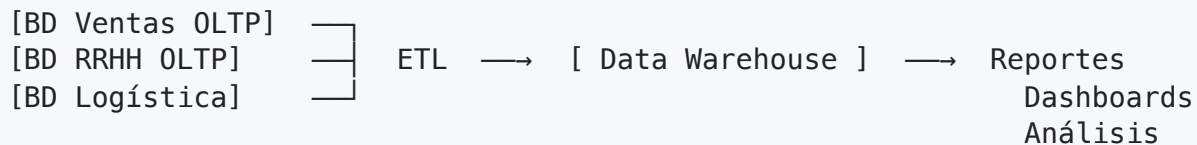
La respuesta habitual: la mayoría de los sistemas reales necesitan **ambos**.

El OLTP registra lo que pasa. El DW responde *qué significa* lo que pasó.

Si estás diseñando una sola BD que debe hacer las dos cosas bien, probablemente deberías estar diseñando dos sistemas.

¿Qué es un Data Warehouse?

- Consolida datos históricos de **múltiples sistemas OLTP**
- Orientado a lectura: las consultas son rápidas, las escrituras son en batch
- No reemplaza la BD transaccional — la complementa
- Los datos llegan al DW mediante un proceso **ETL** (Extract, Transform, Load)



La herramienta de diseño principal para un DW es el **modelo dimensional**.

El Modelo Estrella (*Star Schema*)

Dos tipos de tablas:

Tabla de hechos (*fact table*):

- Contiene las métricas que queremos analizar: montos, cantidades, duraciones
- Una fila por evento del negocio (una venta, una reserva, un clic)
- Muy ancha, millones o miles de millones de filas
- Tiene solo **llaves foráneas + métricas numéricas**

Tablas de dimensión (*dimension tables*):

- Describen el contexto de cada hecho: ¿quién? ¿qué? ¿dónde? ¿cuándo?
- Relativamente pequeñas, bien descriptivas
- **Desnormalizadas** deliberadamente

Las dimensiones rodean la tabla de hechos → el esquema tiene forma de estrella.

Ejemplo: del esquema OLTP al star schema

Objetivo: analizar ventas por cliente, producto, tienda y tiempo.

Tabla de hechos:

```
hechos_ventas(id_cliente, id_producto, id_tienda, id_fecha,  
              cantidad, monto_total, descuento)
```

Solo llaves foráneas + métricas. Sin atributos descriptivos.

Tablas de dimensión:

```
dim_cliente(id_cliente, nombre, ciudad, región, país)  
dim_producto(id_producto, nombre, categoría, familia, precio_base)  
dim_tienda(id_tienda, nombre_tienda, ciudad, región)  
dim_fecha(id_fecha, fecha, día, mes, nombre_mes, trimestre, año,  
           día_semana, es_fin_de_semana, es_feriado)
```

Comparación: OLTP vs. Star Schema

Para saber la región de un cliente:

Antes (OLTP):

Ventas → Clientes → Regiones (2 joins)

Ahora (star schema):

hechos_ventas → dim_cliente (1 join, región está directo)

La consulta de antes (5 joins) se convierte en:

```
SELECT c.región, p.familia, SUM(h.monto_total)
FROM hechos_ventas h
JOIN dim_cliente c ON h.id_cliente = c.id_cliente
JOIN dim_producto p ON h.id_producto = p.id_producto
JOIN dim_fecha f ON h.id_fecha = f.id_fecha
WHERE f.trimestre = 4 AND f.año = 2025
GROUP BY c.región, p.familia;
```

3 joins en lugar de 5, sobre tablas más pequeñas.

Pregunta incómoda

En Semana 5 aprendieron que `ciudad → región` es una dependencia funcional, y que tenerla dentro de `dim_cliente` viola BCNF.

¿No estamos haciendo algo mal?

En `dim_cliente` tenemos:

```
id_cliente → nombre, ciudad, región, país
ciudad      → región      (iDF transitiva!)
región      → país        (iDF transitiva!)
```

En OLTP, normalizaríamos esto. Aquí, deliberadamente no lo hacemos.

Por qué **no** normalizamos las dimensiones:

Argumento	Explicación
Sin anomalías de escritura	Las dimensiones se cargan por ETL, no se modifican fila a fila
Performance	Menos joins → consultas analíticas mucho más rápidas
Legibilidad	Un analista puede leer <code>dim_cliente</code> sin conocer el esquema completo

Por qué **no** normalizamos las dimensiones:

Argumento	Explicación
Sin anomalías de escritura	Las dimensiones se cargan por ETL, no se modifican fila a fila
Performance	Menos joins → consultas analíticas mucho más rápidas
Legibilidad	Un analista puede leer <code>dim_cliente</code> sin conocer el esquema completo

La normalización protege contra anomalías de actualización.

En un DW, las actualizaciones son controladas y masivas (ETL nocturno).

El riesgo que justificaba BCNF no existe aquí.

Regla general: el diseño correcto depende del *workload*. BCNF es correcto para OLTP. El modelo estrella es correcto para OLAP.

Snowflake Schema

Una variante: normalizar las tablas de dimensión.

```
dim_cliente(id_cliente, nombre, id_ciudad)
dim_ciudad(id_ciudad, nombre, id_región)
dim_región(id_región, nombre, id_país)
dim_país(id_país, nombre)
```

- Menos redundancia en las dimensiones
- Más joins para cada consulta
- La mayoría de los practicantes prefiere la estrella

El snowflake schema es literalmente aplicar las reglas de Semana 5 (normalizar a BCNF) al star schema. Es válido, pero en la práctica se evita por la complejidad de las consultas.

ETL: Extract, Transform, Load

El proceso que lleva datos de los sistemas OLTP al DW.

Sistemas OLTP → [Extract] → [Transform] → [Load] → DW

- **Extracción:** conectarse a cada fuente y copiar los datos nuevos o modificados
- **Transformación:** la etapa más compleja — aquí está el trabajo real
- **Carga:** insertar los datos transformados en el DW, típicamente en batch

Frecuencia: generalmente nocturna o de fin de semana.

Transform: aquí esta el sudor

Los datos del mundo real son sucios. Ejemplos típicos:

Problema: el mismo cliente existe en dos sistemas con IDs distintos

```
Sistema A: cliente_id=1042, rut="12.345.678-9", email="juan@ejemplo.cl"  
Sistema B: customer_id=8831, rut="123456789", email="JUAN@EJEMPL0.CL"
```

→ ¿Son la misma persona? Hay que decidir y unificarlos.

Transform: mas sudor

Los datos del mundo real son sucios. Ejemplos típicos:

Problema: valores nulos o inválidos

```
fecha_nacimiento = "32/13/1990" ← fecha imposible  
región = NULL                ← campo obligatorio vacío  
precio = -500                ← precio negativo
```

Transform: sudor

Los datos del mundo real son sucios. Ejemplos típicos:

Problema: encoding inconsistente

```
"Región Metropolitana" / "R.M." / "Santiago" / "13"
```

Todo refiere a lo mismo. El Transform decide el formato canónico.

Transform

El ETL implementa las reglas de negocio que convierten datos crudos en datos confiables:

1. **Deduplicación:** identificar y fusionar registros duplicados
2. **Estandarización:** formatos de fechas, RUTs, nombres de regiones
3. **Validación:** rechazar o corregir valores imposibles
4. **Enriquecimiento:** cruzar con fuentes externas (ej: coordenadas GPS de una dirección)
5. **Agregación:** pre-calculas métricas que se consultan frecuentemente

La calidad del DW depende casi completamente de la calidad del Transform.

"Garbage in, garbage out" es el mantra del data engineering.

Cubo de Datos

Podemos imaginar los datos del DW como un **cubo multidimensional**:

- Cada eje es una dimensión (región, familia, año)
- Cada celda contiene una métrica (total de ventas)

		Electrónica		Ropa	
Norte	[	150k	,	80k	]
Sur	[	120k	,	60k	]
← año 2025					
Norte	[	130k	,	70k	]
Sur	[	100k	,	55k	]
← año 2024					

El cubo es solo una forma de *imaginar* el star schema. Las operaciones OLAP lo navegan.

Roll-up y Drill-down

Roll-up: agregar hacia un nivel más grueso de granularidad

día → mes → trimestre → año	(dimensión tiempo)
producto → categoría → familia	(dimensión producto)
ciudad → región → país	(dimensión cliente)

```
-- De ventas diarias a ventas mensuales
SELECT f.año, f.mes, SUM(h.monto_total)
FROM hechos_ventas h
JOIN dim_fecha f ON h.id_fecha = f.id_fecha
GROUP BY f.año, f.mes;
```

Drill-down: lo inverso — desagregar hacia más detalle.

Ambos se expresan en SQL simplemente cambiando el nivel en el `GROUP BY`.

Slice y Dice

Slice: fijar una dimensión para "cortar" el cubo

```
-- Solo ventas del año 2025 (fijar dimensión tiempo)
WHERE f.año = 2025
```

Dice: filtrar sobre múltiples dimensiones simultáneamente

```
-- Ventas de 2025, región Norte o Sur, solo Electrónica
WHERE f.año = 2025
      AND c.región IN ('Norte', 'Sur')
      AND p.familia = 'Electrónica'
```

Slice y Dice son simplemente cláusulas `WHERE` sobre las dimensiones.

Las operaciones OLAP no son magia nueva — son el vocabulario SQL que ya conocen, aplicado a un modelo dimensional.

Resumen

- Un DW separa las cargas OLTP (escrituras) de las OLAP (análisis)
- El **modelo estrella** organiza los datos en una tabla de hechos + dimensiones desnormalizadas
- La desnormalización viola BCNF deliberadamente — y es la decisión correcta para el workload analítico
- **ETL** lleva los datos al DW; la etapa Transform es donde vive la complejidad real
- Podemos imaginar el modelo como un **cubo** y navegarlo con roll-up, drill-down, slice y dice

Próxima sesión: el SQL que implementa todo esto — ROLLUP, CUBE, window functions — y cómo evolucionó el ecosistema de datos analíticos, del DW tradicional al *lakehouse*.

Data Warehousing

SQL Analítico y el ecosistema de datos

Agenda

1. Recordar GROUP BY — la base
2. ROLLUP: subtotales por jerarquía
3. CUBE: todas las combinaciones
4. Window functions en el contexto analítico
5. Vistas materializadas
6. Una consulta analítica completa
7. Del data warehouse al *lakehouse*: cómo evolucionó el ecosistema

Punto de partida: GROUP BY

Con el star schema del ejemplo, una consulta analítica básica:

```
SELECT c.región, p.familia, SUM(h.monto_total) AS total
FROM hechos_ventas h
JOIN dim_cliente c ON h.id_cliente = c.id_cliente
JOIN dim_producto p ON h.id_producto = p.id_producto
GROUP BY c.región, p.familia
ORDER BY c.región, p.familia;
```

región	familia	total
Norte	Electrónica	150.000
Norte	Ropa	80.000
Sur	Electrónica	120.000
Sur	Ropa	60.000

Útil, pero nos faltan los **subtotales** y el **gran total**.

ROLLUP: subtotales por jerarquía

```
SELECT c.región, p.familia, SUM(h.monto_total) AS total
FROM hechos_ventas h
JOIN dim_cliente c ON h.id_cliente = c.id_cliente
JOIN dim_producto p ON h.id_producto = p.id_producto
GROUP BY ROLLUP(c.región, p.familia)
ORDER BY c.región, p.familia;
```

región	familia	total
Norte	Electrónica	150.000
Norte	Ropa	80.000
Norte	NULL	230.000
Sur	Electrónica	120.000
Sur	Ropa	60.000
Sur	NULL	180.000
NULL	NULL	410.000

NULL en una columna de agrupación = subtotal sobre esa dimensión.

Leyendo la salida de ROLLUP

`ROLLUP(región, familia)` genera **tres niveles** de agrupación:

1. `GROUP BY región, familia` — detalle completo
2. `GROUP BY región` — subtotal por región (`familia = NULL`)
3. (*vacío*) — gran total (`región = NULL, familia = NULL`)

El orden importa: `ROLLUP(A, B) ≠ ROLLUP(B, A)`.

- `ROLLUP(región, familia)` da subtotales por región
- `ROLLUP(familia, región)` daría subtotales por familia

ROLLUP asume una **jerarquía**: el primer elemento es el más grueso, el último es el más detallado.

CUBE: todas las combinaciones posibles

```
GROUP BY CUBE(c.región, p.familia)
```

Genera **todas** las combinaciones de agrupación:

región	familia	total	¿Qué es?
Norte	Electrónica	150.000	detalle
Norte	Ropa	80.000	detalle
Norte	NULL	230.000	subtotal región
Sur	Electrónica	120.000	detalle
Sur	Ropa	60.000	detalle
Sur	NULL	180.000	subtotal región
NULL	Electrónica	270.000	subtotal familia
NULL	Ropa	140.000	subtotal familia
NULL	NULL	410.000	gran total

CUBE agrega las filas de subtotales por familia que ROLLUP omitía.

ROLLUP vs. CUBE

	ROLLUP	CUBE
Agrupaciones generadas	$n+1$	2^n
Asume jerarquía	Sí	No
Caso de uso	Jerarquías claras (año→mes→día)	Análisis multidimensional completo

Con 3 dimensiones:

- `ROLLUP(A, B, C)` → 4 agrupaciones
- `CUBE(A, B, C)` → 8 agrupaciones

Para análisis de tipo "quiero todos los subtotales posibles" → CUBE.

Para "ventas por año, luego por mes, luego por día" → ROLLUP.

Window functions en el contexto analítico

Recordemos window functions de la Semana 6. Aquí se vuelven indispensables.

`GROUP BY` colapsa filas → un resultado por grupo.

Las window functions **agregan sin colapsar** → el resultado tiene la misma cantidad de filas que la entrada.

```
SELECT
  c.región,
  p.familia,
  f.mes,
  SUM(h.monto_total) AS total_mes,
  -- ¿Cuánto representa este mes del total anual de esa región?
  SUM(h.monto_total) * 100.0 /
    SUM(SUM(h.monto_total)) OVER (PARTITION BY c.región) AS pct_region,
  -- Ranking: ¿qué familia vende más en cada región?
  RANK() OVER (PARTITION BY c.región ORDER BY SUM(h.monto_total) DESC) AS rank_familia
FROM hechos_ventas h
JOIN dim_cliente c ON h.id_cliente = c.id_cliente
JOIN dim_producto p ON h.id_producto = p.id_producto
JOIN dim_fecha f ON h.id_fecha = f.id_fecha
GROUP BY c.región, p.familia, f.mes;
```

ROLLUP/CUBE → qué filas de resumen generar.

Window functions → cálculos relativos (porcentajes, rankings, acumulados) sobre el resultado.

Window functions para análisis de series temporales

Patrones que aparecen constantemente en reportes de negocio:

```
SELECT
  f.año, f.mes,
  SUM(h.monto_total)                                AS ventas,
  -- Total acumulado en el año (year-to-date)
  SUM(SUM(h.monto_total)) OVER (
    PARTITION BY f.año ORDER BY f.mes
    ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
  )                                                  AS ytd,
  -- Variación respecto al mes anterior
  SUM(h.monto_total) - LAG(SUM(h.monto_total), 1)
    OVER (ORDER BY f.año, f.mes)                  AS delta_mes,
  -- Media móvil de 3 meses
  AVG(SUM(h.monto_total)) OVER (
    ORDER BY f.año, f.mes
    ROWS BETWEEN 2 PRECEDING AND CURRENT ROW
  )                                                  AS media_movil_3m
FROM hechos_ventas h
JOIN dim_fecha f ON h.id_fecha = f.id_fecha
GROUP BY f.año, f.mes
ORDER BY f.año, f.mes;
```

LAG, acumulados, medias móviles: el pan de cada día en un DW.

Vistas materializadas

Las consultas analíticas sobre el DW son costosas. Si se ejecutan repetidamente, conviene pre-calcularlas.

```
CREATE MATERIALIZED VIEW ventas_por_region_familia AS
SELECT c.región, p.familia, SUM(h.monto_total) AS total
FROM hechos_ventas h
JOIN dim_cliente c ON h.id_cliente = c.id_cliente
JOIN dim_producto p ON h.id_producto = p.id_producto
GROUP BY c.región, p.familia;
```

A diferencia de una vista normal:

- El resultado **se almacena físicamente en disco**
- Las consultas sobre la vista no recalculan nada
- Hay que refrescarla cuando cambian los datos: `REFRESH MATERIALIZED VIEW ventas_por_region_familia;`

Ideal para dashboards y reportes que se consultan muchas veces al día.

Todo junto: una consulta analítica completa

"Ventas de electrónica por región y trimestre en 2025, con subtotales por región"

```
SELECT
  c.región,
  f.trimestre,
  SUM(h.monto_total)      AS total_ventas,
  SUM(h.cantidad)        AS unidades_vendidas,
  AVG(h.monto_total)      AS ticket_promedio
FROM hechos_ventas h
JOIN dim_cliente  c ON h.id_cliente = c.id_cliente
JOIN dim_producto p ON h.id_producto = p.id_producto
JOIN dim_fecha    f ON h.id_fecha    = f.id_fecha
WHERE f.año = 2025
      AND p.familia = 'Electrónica'      -- Slice: solo electrónica
GROUP BY ROLLUP(c.región, f.trimestre)
ORDER BY c.región, f.trimestre;
```

Slice (de la sesión anterior) + ROLLUP + múltiples métricas — en una sola consulta.

Del data warehouse al *lakehouse*

Cómo evolucionó el ecosistema de datos analíticos

El DW tradicional: fortalezas y grietas

Lo que vimos hasta ahora — *star schema* + ETL nocturno sobre un motor relacional (Teradata, Oracle, SQL Server) — fue **la** arquitectura analítica desde los 90 hasta ~2010.

Brilla cuando:

- Los datos son estructurados y el esquema es estable
- Las preguntas se conocen de antemano
- Importan la consistencia, la gobernanza y el tooling BI maduro

El DW tradicional: fortalezas y grietas

Lo que vimos hasta ahora — *star schema* + ETL nocturno sobre un motor relacional (Teradata, Oracle, SQL Server) — fue la arquitectura analítica desde los 90 hasta ~2010.

Brilla cuando:

- Los datos son estructurados y el esquema es estable
- Las preguntas se conocen de antemano
- Importan la consistencia, la gobernanza y el tooling BI maduro

Pero empezó a mostrar grietas:

- **Schema-on-write:** hay que modelar *todo* antes de cargar. Nueva fuente o nueva pregunta → rediseñar esquema y ETL
- **Datos semi/no estructurados** (JSON, logs, texto, imágenes) no caben en un star schema
- **Cómputo y almacenamiento acoplados:** escalar = comprar un *appliance* más grande (\$\$\$)
- La **explosión de volumen** (web, móvil, IoT) desborda ese modelo

¿Dónde sigue vigente el DW tradicional?

El DW relacional **no murió** — dejó de ser la *única* respuesta.

Sigue siendo la mejor opción cuando:

- **Reportería de negocio estable:** finanzas, ventas e indicadores con un esquema bien entendido
- **Cumplimiento y auditoría:** se necesita gobernanza fuerte, trazabilidad y garantías ACID
- **Volúmenes acotados:** no todos somos Google; un Postgres bien diseñado resuelve la mayoría de los casos
- Cuando el tooling BI, las skills SQL del equipo y la consistencia pesan más que la flexibilidad

Importante: el **modelo dimensional** (star schema) **no es obsoleto**.

Sigue siendo el lenguaje común para modelar datos analíticos, incluso *dentro* de los sistemas modernos.

El concepto de data lake: guardar primero, preguntar después

La idea opuesta al schema-on-write: volcar **todos** los datos crudos — en cualquier formato — a almacenamiento de objetos barato (Amazon S3, Google Cloud Storage, MS Azure Blob).

- **Schema-on-read:** la estructura se impone al *consultar*, no al cargar
- Capturas todo *ahora* y decides qué preguntar *después*
- Formatos: CSV/JSON/logs + formatos columnares como **Parquet** y **ORC**
- Almacenamiento y cómputo totalmente desacoplados

El riesgo: sin gobernanza ni catálogo, el lago se vuelve un **data swamp** — datos sin documentar, sin garantías de calidad ni ACID, imposibles de confiar.

El data lake resuelve la rigidez del DW tradicional... a costa del orden que el DW garantizaba.

ETL? ELT!

El data lake invierte el orden del pipeline:

ETL (clásico): Extract → **Transform** → Load

- Transformas *antes* de cargar, en una herramienta aparte
- El esquema se decide por adelantado; cargas datos ya limpios

ELT (moderno): Extract → **Load** → **Transform**

- Cargas los datos crudos primero (al lake o al warehouse)
- Transformas *dentro* del motor, con SQL (ej: **dbt**), on-demand
- Puedes transformar los mismos datos crudos de varias formas, para distintos usos

¿Por qué se invirtió? La nube hizo el almacenamiento casi gratis y el cómputo elástico.

Sale más barato cargar todo crudo y transformar después que adivinar cada pregunta por adelantado.

El Lakehouse: unir el lago y el warehouse

¿Y si tuviéramos la flexibilidad y el costo del lago **con** las garantías del warehouse?

Del data lake	Del data warehouse
Storage barato y abierto (Parquet/S3)	Transacciones ACID
Cualquier tipo de dato	<i>Tenemos un esquema</i>
Cómputo desacoplado y elástico	Gobernanza y versionado

Formatos de tabla abiertos — Delta Lake, Apache Iceberg, Apache Hudi —
añaden transacciones, versionado y *time-travel* sobre archivos Parquet en object storage.

Motores que los consultan directo: Databricks, Spark, Trino, DuckDB, BigQuery/Snowflake (tablas externas).

Es la dirección dominante desde ~2020: un solo lugar para los datos crudos y la analítica gobernada.

Breve historia del OLAP

Año	Hito
1970	Codd formaliza el modelo relacional (nace el dominio OLTP)
1993	Codd acuña el término OLAP ; publica 12 reglas para sistemas analíticos
1993–2000	Primeros DW comerciales: Teradata, Red Brick, Oracle Express; cubos MOLAP en memoria
2000s	Data marts, ETL pesado (Informatica), BI clásico (Business Intelligence Objects)
2006	MapReduce (Google) + Hadoop → DW sobre <i>hardware básico</i>
2012	Amazon Redshift : DW columnar en la nube a escala masiva
2014	Apache Spark: procesamiento distribuido en memoria; desplaza a Hadoop
2016	Snowflake separa almacenamiento y cómputo
2019	dbt y BigQuery ML: el DW como plataforma de transformación
2020–hoy	Lakehouse : unir el lago de datos con el DW. DuckDB para OLAP embebido

Cada salto responde al mismo problema: más datos, más baratos de guardar, más rápidos de consultar.

El ecosistema actual

Sistema	Origen	Diferenciador
Snowflake	2012	Separación cómputo/storage; multi-cloud
BigQuery	Google, 2010	Serverless; pagas por consulta
Redshift	Amazon, 2012	Integración con AWS; columnar
Databricks	2013	Lakehouse: Parquet + Delta Lake + Spark
DuckDB	2019	OLAP embebido; analiza Parquet local sin servidor
ClickHouse	Yandex, 2016	Velocidad extrema en series temporales

A pesar de las diferencias, casi todos comparten los mismos hilos:

- **Almacenamiento columnar** (el *por qué* lo vemos la próxima semana)
- **Separación de cómputo y almacenamiento**
- **SQL sobre object storage**, con escalado elástico / serverless

El star schema sigue siendo el modelo conceptual de referencia — da igual si el motor es Postgres, Snowflake o DuckDB.

Resumen

SQL analítico:

Operación	SQL	Uso
ROLLUP	<code>GROUP BY ROLLUP(A, B)</code>	Subtotales jerárquicos
CUBE	<code>GROUP BY CUBE(A, B)</code>	Todas las combinaciones
Window functions	<code>... OVER (PARTITION BY ...)</code>	Agregar sin colapsar filas
Vista materializada	<code>CREATE MATERIALIZED VIEW</code>	Pre-calcular consultas costosas

La evolución del ecosistema:

- DW tradicional (schema-on-write, ETL) → data lake (schema-on-read, ELT) → **lakehouse**
- El DW relacional sigue vigente para reportería estable y gobernada
- El **modelo dimensional** sobrevive a todos los motores

¿Por qué el DW es lento de todas formas?

Incluso con el star schema, las consultas sobre millones de filas son lentas porque:

- La tabla de hechos se almacena **por filas** (NSM): leer `SUM(monto_total)` requiere leer páginas enteras aunque solo necesitemos una columna
- Los joins con las dimensiones, aunque menores, siguen siendo costosos a escala

Próxima clase: Bases de datos columnares.

